VSF INTERN · TRACK MLE · DỰ ÁN GEOLIFE HOME / OFFICE

Tổng kết thực tập lần 2

Người trình bày: Trần Chí Tâm 20/10/2026

AGENDA

Nội dung

01 **Bài toán và dữ liệu**
Pipeline từ GPS thô đến Home / Office · làm sạch · stay-point

02 **Classifier v1 và v2**
Chấm điểm từng stay · gom địa điểm bằng DBSCAN · so sánh

03 **Least Privilege trên GCP**
Phân quyền tối thiểu (Least Privilege) khi đưa lên Cloud Run

04 **Tổng kết**
Đã làm và bài học

01 · BÀI TOÁN VÀ DỮ LIỆU

Suy luận nhà và cơ quan từ quỹ đạo GPS

Từ GPS thô của một người, tìm **Home**, **Office** và các địa điểm hay đến (POI), trả về qua API.

GPS THÔ
24.88 M
điểm · 182 user · 18,670 file `.plt` · giờ GMT

→

LÀM SẠCH
24.18 M
795 điểm đưa vào quarantine kèm lý do

→

STAY-POINT
18,597
ở yên ≥ 30 phút trong 200 m · 174 user

→

CLASSIFIER + API
v2
DBSCAN · home 114 / office 108 user

Dữ liệu Microsoft GeoLife **không có nhãn nhà hay cơ quan**. Nhãn chỉ là phương tiện di chuyển và phủ 20.1% số điểm.

Không có đáp án để đối chiếu, nên mỗi bước làm sạch đều có **notebook bằng chứng**; dòng bị loại không bị xoá âm thầm mà ghi vào **quarantine** kèm lý do.

Mỗi ngưỡng xử lý đều có bằng chứng từ dữ liệu

QUYẾT ĐỊNH	BẰNG CHỨNG (DỮ LIỆU THẬT)	KẾT QUẢ
Ngưỡng tốc độ bất khả thi	Điểm thật nhanh nhất 1,048 km/h; lỗi GPS chậm nhất 1,123 km/h	**1,100 km/h**
Cắt đoạn khi mất tín hiệu	Khoảng ngắt trong một chuyến: P99.9 = 252 s, P99.99 = 2,170 s; không có điểm gãy tự nhiên, ở 20 phút chỉ 0.02% khoảng ngắt trong chuyến bị cắt nhầm	**20 phút (quy ước)**
Nối các file của một user	Tỉ lệ "ghi lại ở cùng chỗ" chạm mức nền 19.7% ở 18 h; số đêm tìm được 395 → 3,340	**≤ 18 h**
Giờ địa phương	2.61% số điểm ngoài UTC+8; UTC+8 cố định đặt 21.5% hoạt động vào 1–5 h sáng	**Theo toạ độ**
Bán kính / thời gian stay-point	Ngưỡng phổ biến trong các nghiên cứu trên GeoLife (Li et al. 2008; Zheng et al.), kiểm tra bằng cách xem stay-point trên bản đồ	**200 m / 30 phút (tham khảo)**

02 · CLASSIFIER V1

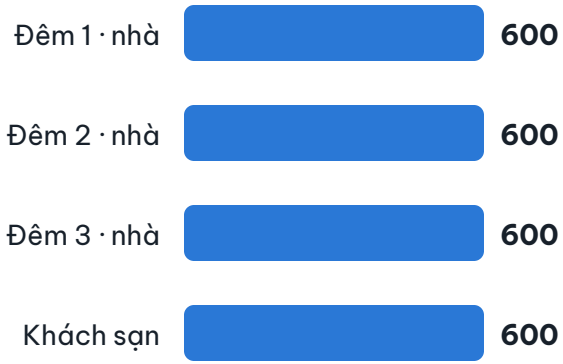
v1: mỗi stay tự chấm điểm, stay nhiều phút nhất thắng

1 Các stay của một user



mỗi chấm = 1 stay · số = phút trong khung 22–08

2 Chấm điểm từng stay



4 stay hoà nhau

3 Kết quả

- chọn 1 stay (luật phá hoà)
- ≥ 240 phút trong khung
- có stay khác trong 200 m

HOME

Home 22–08 h · office 09–18 h T2–T6 (giờ địa phương) · chấm riêng từng loại · không đạt cổng thì **unknown**

Trả lời được (home / office)

164 / 159 → 104 / 100

Classifier v1 · notebooks 12, 13

Giới hạn của v1: nhiều đêm hoà điểm nhau

Ví dụ: một user ở nhà 3 đêm và khách sạn 1 đêm

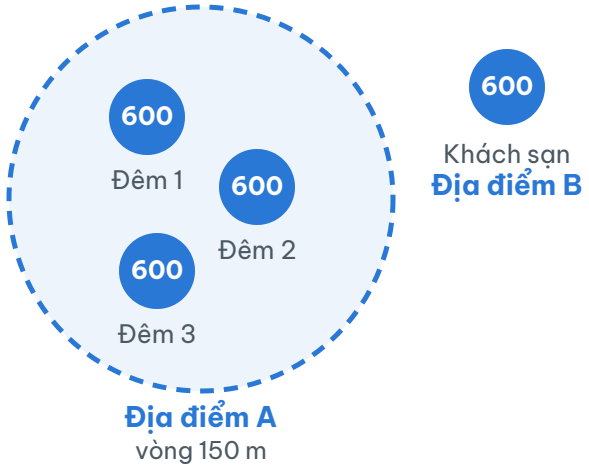
STAY	Ở ĐÂU	PHÚT TRONG KHUNG 22–08
Đêm 1	Nhà	600
Đêm 2	Nhà	600
Đêm 3	Nhà	600
Đêm 4	Khách sạn	600

> Một đêm trọn đã lấp đầy khung (600 phút là tối đa), nên bốn stay **hoà nhau** và v1 chỉ chọn một theo luật phá hoà.

> v1 **không gộp** các stay gần nhau, nên không biết ba đêm đầu cùng một chỗ, và có thể chọn khách sạn.

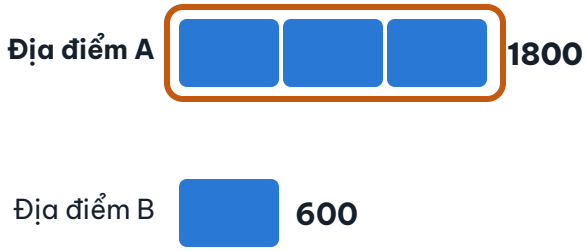
v2: gom các stay gần nhau thành địa điểm, rồi cộng phút

1 Các stay của một user



mỗi chấm = 1 stay · số = phút trong khung 22–08

2 Cộng phút theo địa điểm



A thắng, không còn hoà

3 Kết quả

- tổng 1800 ≥ 240 phút
- 3 ngày ≥ 2 ngày
- vị trí = tâm địa điểm

HOME

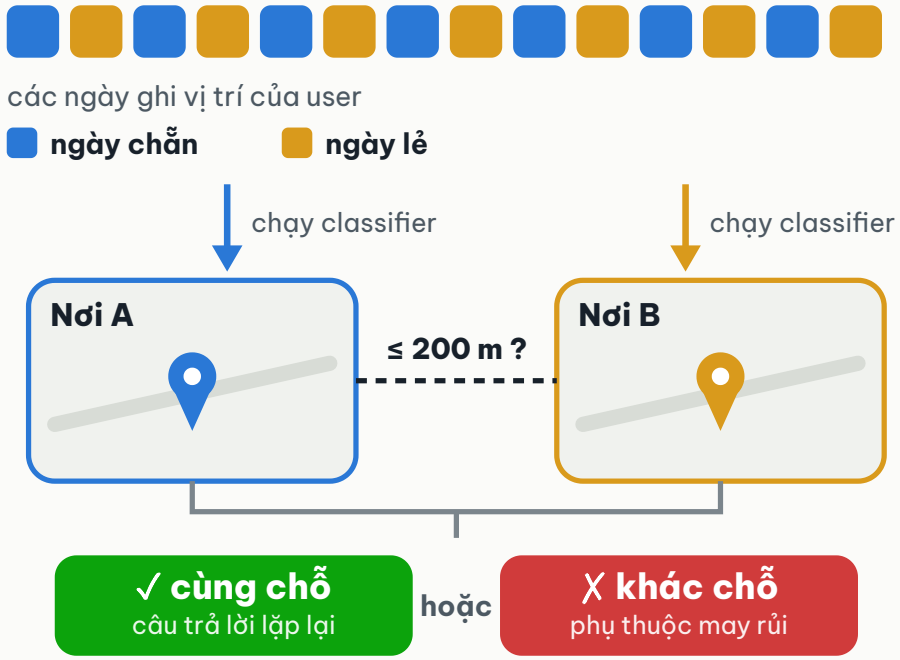
Gom bằng DBSCAN, bán kính **150 m** · stay lẻ vẫn là một địa điểm riêng · cùng khung giờ như v1

Trả lời được (home / office)

114 / 108

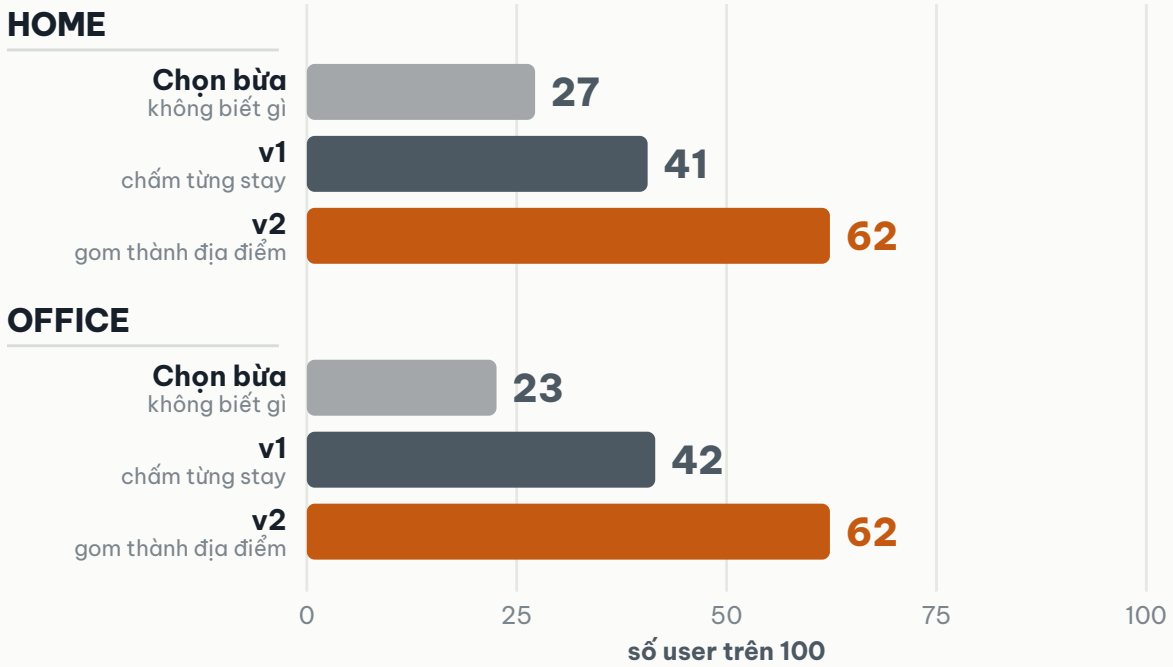
v2 cho câu trả lời lặp lại được nhiều hơn v1

Cách đo, cho 1 user



Làm riêng cho home và cho office, trên 100 user.

Trong 100 user, bao nhiêu user có hai nửa cho cùng một chỗ?



Đọc: càng cao thì câu trả lời càng ít phụ thuộc may rủi. Đây là độ **lặp lại**, chưa phải độ đúng (không có nhãn để kiểm tra). Chia theo thời gian (có cả chuyển nhà thật) thì mọi số thấp hơn, thứ tự vẫn vậy.

Least Privilege: chỉ cấp đúng quyền cần để làm việc

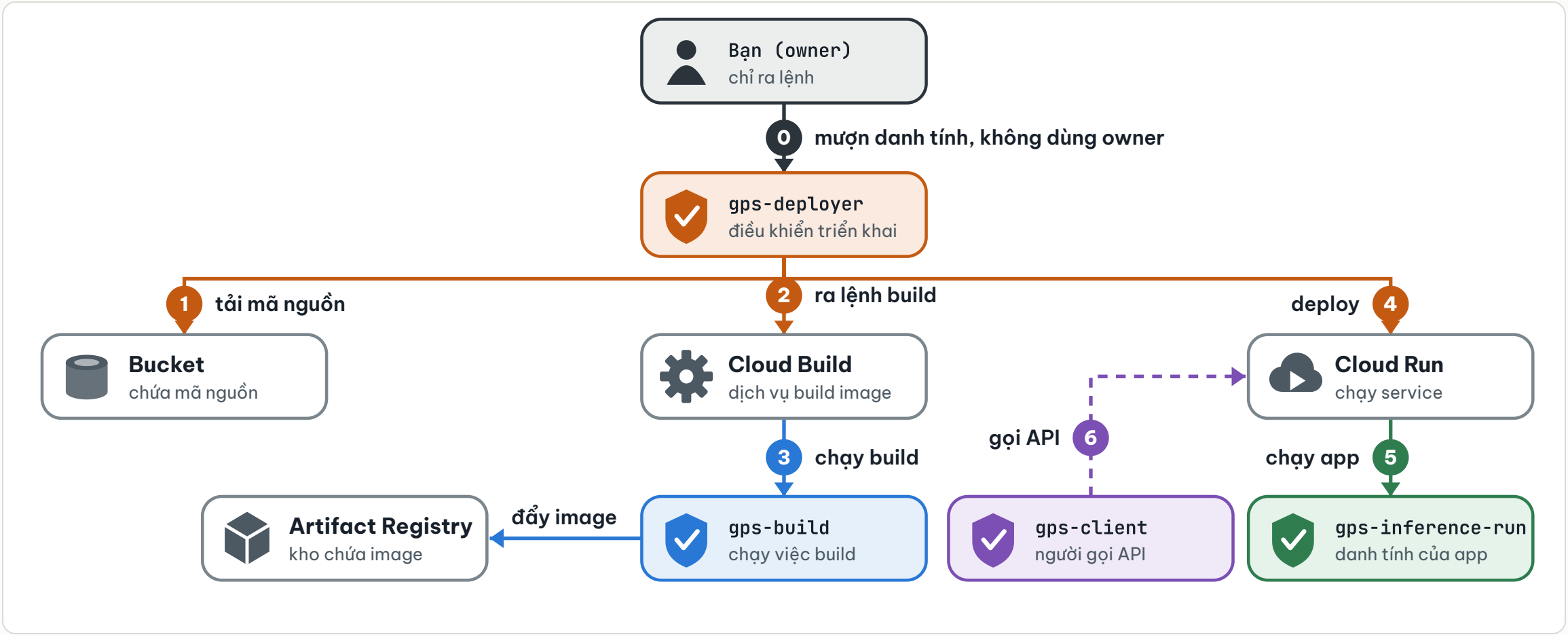
Nguyên tắc

- Chỉ cấp quyền **vừa đủ** để làm đúng việc của nó, không hơn
- Gắn quyền vào **đúng tài nguyên** (một service, một repo), không gắn cho cả project
- Mỗi việc dùng **một danh tính riêng**, không dùng chung một tài khoản cho mọi thứ

Vì sao cần

- Không hệ thống nào an toàn tuyệt đối: một thành phần có thể bị chiếm hoặc cấu hình sai
- Khi đó kẻ tấn công chỉ có **quyền của thành phần đó**. Quyền càng hẹp, thiệt hại càng nhỏ

Quy trình deploy: ai làm gì ở mỗi bước



Bước 0–1: mượn deployer, rồi đưa mã nguồn lên bucket

Ai thực hiện: bạn → gps-deployer

Việc cần làm: mượn danh tính deployer thay vì dùng quyền owner, rồi tải mã nguồn (nén) lên bucket trên cloud.

ROLE / QUYỀN	NGHĨA LÀ GÌ	GẮN Ở ĐÂU
`serviceAccountTokenCreator`	Cho phép bạn mượn danh tính `gps-deployer`, không cần tải key	trên gps-deployer
`storage.objectAdmin`	Tạo, đọc, sửa, xoá **file** trong bucket (CRUD)	chỉ bucket vsfintern_cloudbuild
`storage.legacyBucketReader`	Xem thông tin bucket (gcloud cần để tìm nơi tải lên)	cùng bucket
`serviceusage.serviceUsageConsumer`	Được phép gọi API của project (tính vào project)	project

Vì sao không dùng owner: owner làm được mọi thứ nên quá rộng. Mượn deployer thì mọi lệnh chỉ mang quyền hẹp, và không có key nào nằm lại trên máy.

Bước 2–3: người ra lệnh build khác người chạy build

Bước 2 · ra lệnh do gps-deployer

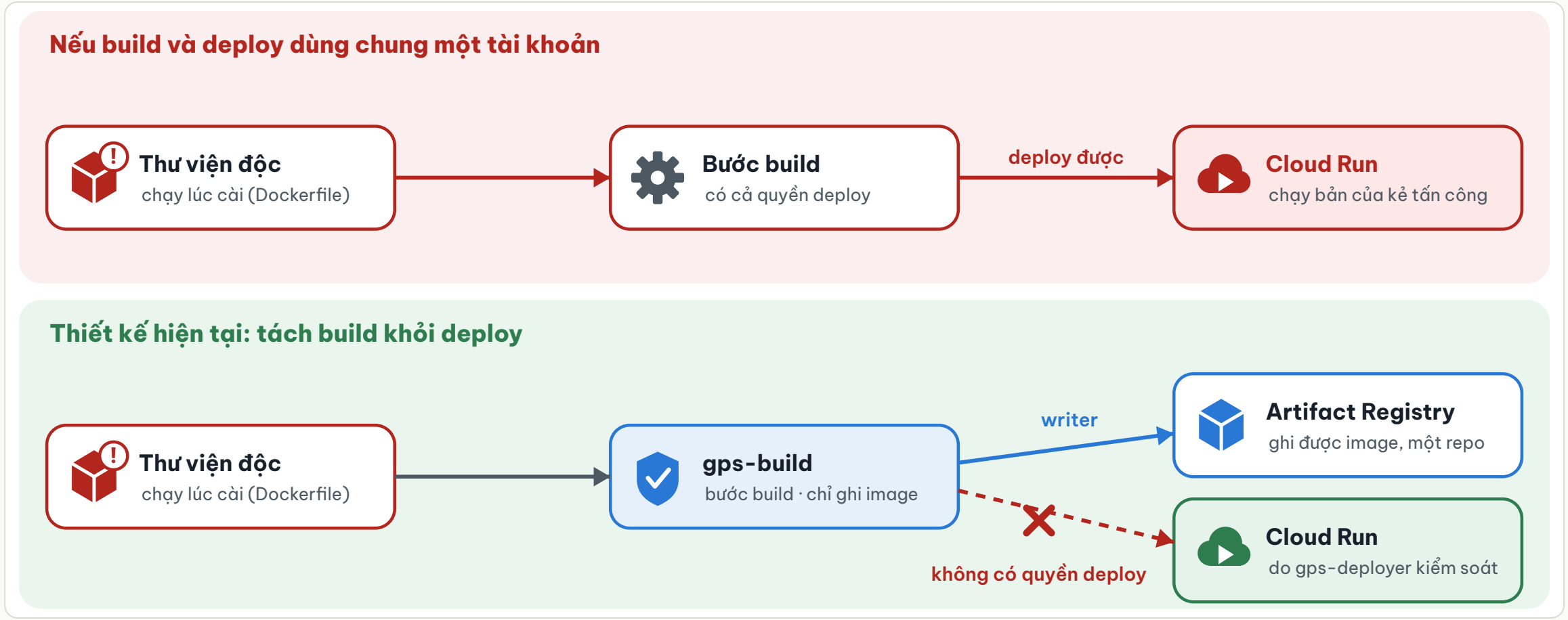
ROLE / QUYỀN	NGHĨA LÀ GÌ	GẮN Ở ĐÂU
`cloudbuild.builds.editor`	Tạo và quản lý một lần build	project
`serviceAccountUser (actAs)`	Chỉ định build chạy bằng `gps-build`	chỉ trên gps-build

Bước 3 · chạy build do gps-build

ROLE / QUYỀN	NGHĨA LÀ GÌ	GẮN Ở ĐÂU
`storage.objectViewer`	Chỉ **đọc** file mã nguồn	bucket source
`artifactregistry.writer`	Ghi image vào kho	chỉ repo gcr.io
`logging.logWriter`	Ghi log của lần build	project

Hai bộ quyền không trùng nhau. Ví dụ: quản lý (deployer) giao việc cho nhân viên (`gps-build`) nên cần quyền giao việc và quyền nhân danh nhân viên. Nhân viên cần công cụ để làm: đọc file, ghi image, ghi log.

Vì sao build và deploy phải là hai danh tính khác nhau



Bước 4: deploy image lên Cloud Run

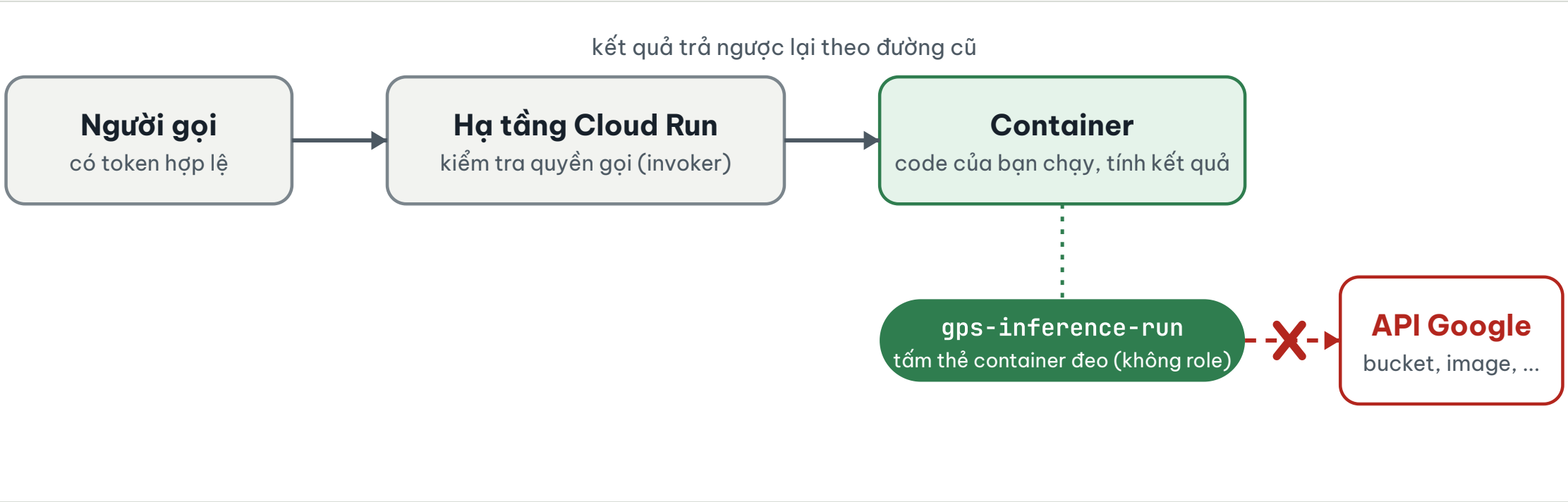
Ai thực hiện: `gps-deployer`

Việc cần làm: tạo revision mới của service từ image vừa build, và chỉ định service sẽ chạy bằng tài khoản `gps-inference-run`.

ROLE / QUYỀN	NGHĨA LÀ GÌ	GẮN Ở ĐÂU
`artifactregistry.reader`	Đọc image để chắc nó tồn tại trước khi deploy	chỉ repo gcr.io
`run.developer`	Tạo revision, đổi cấu hình service. **Không gồm** quyền sửa IAM của service	chỉ service gps-inference
`serviceAccountUser (actAs)`	Chỉ định service chạy bằng `gps-inference-run`	chỉ trên gps-inference-run

Một cơ chế bảo vệ có thật: khi deploy, gcloud cố đặt lại IAM của service để mở công khai. Deployer không có quyền sửa IAM nên bị từ chối, service vẫn private.

Bước 5: chạy app, tài khoản runtime chỉ là tấm thẻ



Không phải trung gian. Tài khoản `gps-inference-run` không nhận hay chuyển request nào. Container mới là nơi code chạy.

Chỉ như tấm thẻ. Thẻ chỉ dùng khi container muốn mở cửa một API Google. App không gọi API nào nên thẻ **không cần role**. Nếu code bị chiếm, kẻ tấn công cầm tấm thẻ trống.

Bước 5: quyền của gps-inference-run

Ai thực hiện: `gps-inference-run`

Việc cần làm: chạy app khi có request. Danh tính này cần quyền gì, và ai được dùng nó.

ROLE / QUYỀN	NGHĨA LÀ GÌ	GẮN Ở ĐÂU
`không có role nào`	Không được gọi API Google nào, chỉ làm danh tính của container	không gắn ở đâu
`serviceAccountUser (actAs)`	Cho phép **gps-deployer** chỉ định service chạy bằng tài khoản này	trên gps-inference-run

Vì sao không cấp role: app không gọi API Google nào (cấu hình nằm trong image, log ra stdout) nên mọi quyền đều thừa. Nếu container bị chiếm, kẻ tấn công không lan ra được.

Bước 6: gọi API, quyền invoker và vì sao service private

Service là API private

- Mọi request phải kèm **ID token** do Google cấp, nếu không là **403**
- `run.invoker` = quyền **gọi** (invoke) service này, gắn đúng trên service
- Lý do private: stay-point là **vị trí cá nhân**, nên chặn mặc định và chỉ mở cho ai cần

Ai gọi trong thực tế

- Người dùng cuối **không** gọi thẳng. Họ đi qua một ứng dụng phía trước (web, backend)
- Ứng dụng đó mang quyền **invoker** để gọi service
- `gps-client` là danh tính **minh hoạ và kiểm chứng**: bạn mượn nó để thử gọi từ máy mình

Bước 6: quyền của gps-client

Ai thực hiện: `gps-client`

Việc cần làm: gọi API của service. Danh tính này cần quyền gì, và ai được dùng nó.

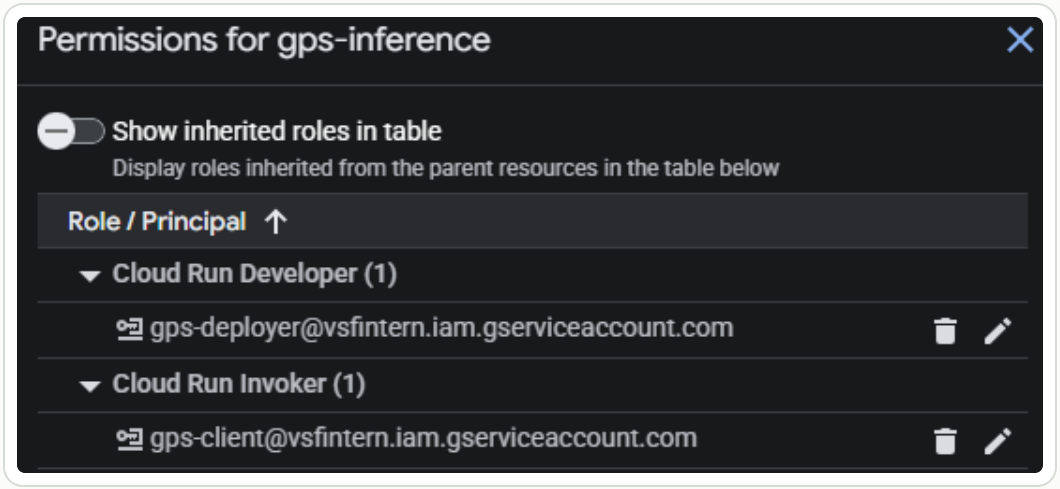
ROLE / QUYỀN	NGHĨA LÀ GÌ	GẮN Ở ĐÂU
`run.invoker`	Được **gọi** (invoke) service Cloud Run private, không có thì nhận 403	chỉ service gps-inference
`serviceAccountTokenCreator`	Cho phép **bạn (owner)** mượn `gps-client` để gọi thử và kiểm chứng	trên gps-client

Quyền gắn với danh tính: một người có tài khoản Google nhưng không có `run.invoker` vẫn nhận 403. Trong thực tế server của ứng dụng chạy bằng `gps-client` sẽ gọi API; ở đây mượn nó để kiểm chứng.

Bước 6: bằng chứng từ Cloud Run console



Cloud Logging, request vào /health: **403** gọi không token (curl) · **401** token rỗng hoặc sai · **200** token của `gps-client` (dòng cuối, 16:34 giờ ICT).



Permissions của service: chỉ hai dòng, đúng thiết kế.

```
PS> Invoke-RestMethod -Uri "$url/health"
     -Headers @{ Authorization = "Bearer $token" }

status  version model_version
------  ------- -------------
healthy 0.1.0   v2
```

Kết quả lệnh gọi với token của `gps-client`.

Đã làm được gì và học được gì

Đã làm

- Pipeline làm sạch: 24.18 M điểm, 795 quarantine có lý do
- 18,597 stay-point, 9 tính chất kiểm 0 vi phạm
- v1 và v2, đo bằng độ lặp lại 41% → 62%
- API FastAPI, 156 test, 15 notebook bằng chứng
- Cloud Run: shadow 174 user, promote v2, phân quyền tối thiểu

Bài học

- Mỗi ngưỡng cần bằng chứng, không chọn tuỳ ý
- Không có nhãn thì đo cái đo được (độ lặp lại) và nói rõ nó chưa phải độ đúng
- **Quyền tối thiểu:** hỏi "app thật sự cần gì" trước khi cấp; một việc một danh tính; không key dài hạn; gỡ quyền thừa

Kết quả Least Privilege: nếu container bị chiếm

- Không đọc hay ghi được bucket, image, log; không deploy, không sửa được IAM
- Không gọi được API Google nào, vì tài khoản runtime **không có role**; service vẫn private (không token thì 403)
- Đã kiểm chứng trên Cloud Run: revision `gps-inference-00008` chạy bằng `gps-inference-run`; token của `gps-client` gọi `/health` trả `v2`

Q&A

Mời câu hỏi

Cảm ơn đã lắng nghe

VSF Intern · Track MLE · Trần Chí Tâm